Customer Shopping Behavior Analysis

1. Project Overview

This project examines customer shopping behavior using transactional data from 3,900 purchases across multiple product categories. The objective is to identify insights related to spending patterns, customer segments, product preferences, and subscription trends in order to support informed strategic business decisions.

2. Dataset Summary

-Rows: 3,900

- Columns: 18

- Key Features:

- Customer Demographics: Age, gender, location, and subscription status.
- Purchase Details: Item purchased, product category, purchase amount, season, size, and color.
- Shopping Behavior: Discount usage, promo code usage, number of previous purchases, purchase frequency, review ratings, and shipping type.
- Missing Data: 37 missing values identified in the Review Rating column.

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

- Data Loading: Imported the dataset using pandas.
- Initial Exploration: Used df.info() to check structure and .describe() for summary statistics.

```
[73]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

```
[74]: df.describe(include = 'all')
```

[74]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN

- Missing Data Handling: Identified null values and filled the missing entries in the Review Rating column by imputing the median rating for each respective product category

```
75]: df.isnull().sum()
```

```
75]: Customer ID               0
     Age                       0
     Gender                    0
     Item Purchased            0
     Category                  0
     Purchase Amount (USD)     0
     Location                  0
     Size                      0
     Color                     0
     Season                    0
     Review Rating            37
     Subscription Status       0
     Shipping Type             0
     Discount Applied          0
     Promo Code Used           0
     Previous Purchases        0
     Payment Method            0
     Frequency of Purchases    0
     dtype: int64
```

```
[76]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x : x.fillna(x.median()))
```

```
[57]: df.isnull().sum()
```

```
[57]: Customer ID               0
      Age                       0
      Gender                    0
      Item Purchased            0
      Category                  0
      Purchase Amount (USD)     0
      Location                  0
      Size                      0
      Color                     0
      Season                    0
      Review Rating             0
      Subscription Status       0
      Shipping Type             0
      Discount Applied          0
      Promo Code Used           0
      Previous Purchases        0
      Payment Method            0
      Frequency of Purchases    0
      dtype: int64
```

- Column Standardization: Renamed column headers using snake_case format to improve readability and maintain consistent documentation.

- **Feature Engineering**:
  - Created age_group column by binning customer ages.
  - Created purchase_frequency_days column from purchase data.

[78]:
```
# Create a column age_group
labels = ['Young Adult','Adult','Middle-Age','Senior']
df['age_group'] = pd.qcut(df['age'],q =4 , labels = labels)
```

[62]:
```
df[['age','age_group']].head(10)
```

[62]:

	age	age_group
0	55	Middle-Age
1	19	Young Adult
2	50	Middle-Age
3	21	Young Adult
4	45	Middle-Age
5	46	Middle-Age
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-Age

[79]:
```
# Create column purchase_frequency_days
frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}
df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

[80]:
```
df[['purchase_frequency_days','frequency_of_purchases']].head(10)
```

[80]:

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

- **Data Consistency Check**: Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

```
[81]: df[['discount_applied','promo_code_used']].head(10)
```

[81]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[82]: (df['discount_applied'] == df['promo_code_used']).all()
```

```
[82]: np.True_
```

- **Database Integration:** Connected Python script to MySQL and loaded the cleaned DataFrame into the database for SQL analysis.

4 . Data Analysis using SQL (Business Transactions)

We performed structured analysis in PostgreSQL to answer key business questions:

1 **Revenue by Gender** – Compared total revenue generated by male vs. female customers.

Results | Messages

	gender	revenue
1	Male	157890
2	Female	75191

2 **High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

Results | Messages

	customer_id	purchase_amount
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62
11	24	88
12	29	94
13	32	79
14	33	67
15	35	91
16	37	69
17	40	60
18	41	76
19	43	100
20	44	69
21	52	59
22	55	94
23	57	73
24	58	64
25	60	79
26	62	68
27	64	79
28	65	83
29	67	94

Query executed successfully.

3 **Top 5 Products by Rating** – Found products with the highest average review ratings.

Results | Messages

	item_purchased	average product rating
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.8
5	Skirt	3.78

4 **Shipping Type Comparison** – Compared average purchase amounts between Standard and Express shipping.

Results | Messages

	shipping_type	(No column name)
1	Standard	58
2	Express	60

5 **Subscribers vs. Non-Subscribers** – Compared average spend and total revenue across subscription status.

Results | Messages

	subscription_status	total_customers	avg_spend	total_revenue
1	Yes	1053	59	62645
2	No	2847	59	170436

6 **Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

Results | Messages

	item_purchased	discount_rate
1	Hat	50
2	Coat	49
3	Sneakers	49
4	Sweater	48
5	Pants	47

7 **Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

Results | Messages

	customer_segment	Number of Coustomer
1	Returning	701
2	Loyal	3116
3	New	83

8 **Top 3 Products per Category** – Listed the most purchased products within each category.

Results | Messages

	item_rank	item_purchased	total_orders
1	1	Jewelry	171
2	2	Belt	161
3	3	Sunglasses	161
4	1	Blouse	171
5	2	Pants	171
6	3	Shirt	169
7	1	Sandals	160
8	2	Shoes	150
9	3	Sneakers	145
10	1	Jacket	163
11	2	Coat	161

9 **Repeat Buyers & Subscriptions** – Checked whether customers with >5 purchases are more likely to subscribe.

	subscription_status	repeat_buyers
1	Yes	958
2	No	2518

10 **Revenue by Age Group** – Calculated total revenue contribution of each age group.

Results | Messages

	age_group	total_revenue
1	Young Adult	62143
2	Middle-Age	59197
3	Adult	55978
4	Senior	55763

5. Dashboard in Power BI

Finally, we built an interactive dashboard in Power BI to present insights visually.

6 Business Recommendations:

- **Increase Subscriptions** – Promote exclusive benefits and incentives to encourage more customers to subscribe.
- **Strengthen Customer Loyalty Programs** – Reward repeat customers to help move them into the "Loyal" segment.
- **Review Discount Strategy** – Ensure discounts drive sales while maintaining healthy profit margins.
- **Improve Product Positioning** – Feature top-rated and best-selling products prominently in marketing campaigns.

- **Implement Targeted Marketing** – Direct marketing efforts toward high-revenue age groups and customers who prefer express shipping.